

achieve the same results.

Decentralised governance: The monolithic application standardises on a single technology platform which may seem healthy to start with but, later, it may become a bottleneck when some of the modules or components need to use other styles.

In Microservices, each team can select a different standard for the service it is building. Netflix is a good example.

Decentralised data management: In Microservices, instead of using the same database, each service can decide its own storage. For example, LinkedIn was using Oracle when it was running as a monolithic application, but then moved to Microservices and started using the graph database for storing people and their connections, apart from Oracle. Microservices prefers that each service manages its own database.

Designed to cope with failure: When we think of service as a component, the application on the whole is to be designed to cope with the failure of the service. As this is mostly going to be a network call using an end point which is bound to fail, it should be reflected upon. Netflix uses SimianArmy to test failure.

Evolutionary design: When we think of Microservices, we think in terms of decomposing the system into small services. The key part is the notion of independent replacement and upgradeability. The Guardian website is moving towards Microservices, due to which it can

easily add quick new sections for new events which can be removed quickly once the event is over, as it is a service.

To get a clearer picture, look at Figure 6, which shows a conceptual monolithic application and Figure 7, which depicts the application split into Microservices.

The Microservices approach has some drawbacks like dealing with the complexity of a distributed system. The two common points of failure are services failing to respond and high latency. Transaction management in distributed systems is difficult to maintain. It brings lot of operational overhead, and the team should be ready to manage and monitor a lot of services which are redeployed continuously. If the organisation does not have the Dev-Ops culture, moving towards Microservices is bound to fail. 

References

- [1] <http://www.martinfowler.com/articles/microservices.html>
- [2] <http://microservices.io/>

By: Ashish Singh Bhatia

The author is a technology enthusiast and a FOSS fan. He loves to explore new technology, and work on Python and Java languages. He can be reached at ast.bhatia@gmail.com and blogs at <https://openfreeidea.wordpress.com/>

Continued from page 77...

```
$ git cherry-pick -m master <commit-id>
/* Test again in release version to make sure */
/* Push to official-repository */
```

Note that since we had squashed multiple commits into a single logical commit for the defect concerned, as part of our development workflow, only a single commit needed to be cherry-picked. If *squash* was not done, we would have had to track and cherry-pick multiple commits.

Common errors while using Git and how to recover from them

There are times when we make mistakes, but the power of Git helps us to recover from them. However, for the following actions, do read up the documentation and understand what they do and what the side-effects could be, before use.

- To undo a wrong change in a file which is not committed, use the command given below:

```
$ git checkout -- <file>
```

- If you missed committing a file from a commit that you just made, issue the following command:

```
$ git add <missed-file>
$ git commit -amend
```

- If you wrongly committed some change, use the command shown below:

```
$ git reset HEAD~1
```

- If you messed up and want to go to a clean slate without having to clone again (i.e., you don't want generated or untracked files around), use the following code:

```
$ git clean -x -f
```



References

- [1] *Git Magic*: <http://www-cs-students.stanford.edu/~blynn/gitmagic/>
- [2] *Pro Git*: <http://git-scm.com/book>
- [3] *Online reference*: <http://git-scm.com/docs>
Book: 'Version Control with Git', Jon Loeliger and Matthew McCullough, O'Reilly Publishers.

By: Natarajan Venkataraman

The author is an industry veteran with over two decades of professional experience in embedded systems and networking domains. His areas of interest include quality of service, security, algorithms and last, but not the least, good software development practices. He can be reached at v_natarajan@acm.org.